
CHAPTER 03

AI 시대의 개발 방법론

왜 지금 방법론

시기	현상	대표 키워드/도구	핵심 질문
2025.02	AI 코딩의 대중화와 문화적 전환점	Vibe Coding	일단 빨리 만들어 보자
2025 중반	생성 코드의 품질, 보안, 유지보수 문제 부각	구조화된 검토, 검증 루프	이 결과를 믿어도 되나?
2025 하반기	구현 전에 방향을 고정하려는 업계 대응	SDD, Spec-first, GitHub Spec Kit	애초에 무엇을 만들라고 할 것인가?
2026 초	구현 뒤 검증 체계와 문맥 통제가 다시 중요해짐	TDD, Context Engineering	정말 그 스펙대로 동작하는가?

AI에게 무엇을 시킬지, 어떻게 검증할 것인지

SDD

Spec-Driven Development

SPECIFY

/speckit.specify

WHAT / WHY 분리

[NEEDS CLARIFICATION]

PLAN

/speckit.plan

HOW 제안

Constitution Check

TASKS

/speckit.tasks

태스크 분해

병렬 가능 작업 [P] 마킹

강제되는 원칙

WHAT/WHY와 HOW를 분리하고, 모호한 부분은 [NEEDS CLARIFICATION]에서 멈춘다.

핵심

모든 산출물이 파일로 저장되고 커밋되고 계속 참조된다.



GitHub Copilot icon GitHub spec-kit

TDD (Test-Driven Development)

테스트를 먼저 쓰고, 통과하는 코드를 나중에 쓴다. AI 시대에는 인간이 테스트, AI가 구현.

Red

인간이 실패 테스트 작성

Green

AI가 통과 코드 구현

Refactor

인간 리뷰 → AI가 리팩토링

왜 AI에게 특히 중요한가

AI는 "동작하는 것 같은" 코드를 자신 있게 만들. 테스트 없으면 맞는지 틀린지 확인 불가.

AI의 치팅에 주의

테스트 수정 금지 규칙 필수. 테스트 코드 임의 수정 금지 · assert 조건 약화 · 실패 확인 전 구현 금지.

Waterfall vs SDD

	Waterfall	SDD
개발 흐름	요구사항 → 설계 → 코딩 → 테스트	스펙이 primary artifact
검증 시점	테스트가 개발 후반에 실제 제약을 드러냄	/speckit.specify → /speckit.plan → /speckit.tasks
실행 산출물	요구사항 또는 설계를 다시 고침	spec · plan · tasks를 실행 산출물로 갱신

SDD + TDD가 Harness로 이어지는 이유

Spec

스펙 템플릿

계획 문서



TDD

TDD 루프

Skills

Hooks

이 시스템이 곧 하네스 엔지니어링